The Category of Containers

a living reference

MacBeth
Kodamai AI Mathematician project

Draft, June 2026, v0.1

Contents

Preface iii

1 Containers and their extension functor 1
- 1.1 Containers . . . 1
- 1.2 The representation theorem . . . 1

2 The category Cont 3
- 2.1 Container morphisms: the variance . . . 3
- 2.2 Worked example: `tail` . . . 3

3 Algebraic structure on Cont 5
- 3.1 Products and coproducts . . . 5
- 3.2 The monoidal menagerie . . . 5

4 Comonoids in Cont: directed containers and small categories 7
- 4.1 Directed containers . . . 7
- 4.2 Directed containers are comonads . . . 7
- 4.3 Directed containers are small categories . . . 8
- 4.4 The morphism refinement: **DCont** $\cong$ **Cof** . . . 8

5 Composing systems: Zappa–Szép and distributive laws 11
- 5.1 The pairwise criterion . . . 11
- 5.2 (G) as a strict factorisation system, and its H^2 obstruction . . . 11
- 5.3 The classical anchor . . . 12

6 Functors over the category of containers (Phase 2 — outline) 13

A Reading & citation tracker 15

Preface

Purpose of this document

This is a single, evolving reference centred on *the category of containers*. It is meant to grow: chapters are added and revised in place, and the whole document compiles to one PDF that can be read end to end or consulted by section. The organising thesis is the equivalence chain

Containers $\simeq$ **Directed Containers** $\simeq$ **Polynomial Comonoids** $\simeq$ **Small Categories**,

which we take as the foundation for *verified composition*: once a system (a supply chain, an agent hierarchy, a state space, a tax computation) is presented as a container, its composition is governed by the structure of **Cont** and of the comonoids inside it, and correctness of composition becomes a statement that can be proved — and, increasingly, machine-checked. Object level equivalence is settled groundwork; the live mathematics lives at the *morphism* level, and that is where this book spends most of its effort.

The flagging convention

Mathematical honesty is the first principle of this document. Every **Definition** and every result is flagged, and every **Theorem**/**Proposition**/**Lemma**/**Corollary** statement *ends with a bracketed provenance tag*, one of:

- [Cited: *author year, ref*] — a known result from the literature, not claimed as new;
- [MacBeth] — claimed and/or proved by MacBeth, informally (not machine-checked);
- [MacBeth, Lean-verified: *file*] — machine-checked in the repository's `lean/` tree, in the named source file;
- [Conjecture, MacBeth] or [Open] — for conjectures.

A tag [Cited: . . .] on a statement that MacBeth has also verified independently will say so; when in doubt the weaker (cited, not-checked) tag is used, and a footnote records the uncertainty. The bias is deliberately towards *under*-claiming. The reading and citation tracker in Appendix A records, paper by paper, how thoroughly each source has been engaged.

A note on the Lean tags. Only two Lean *source* files are committed to the repository at the time of this draft: `Basic.lean` (the container representation theorem) and `Directed.lean` (the comonad laws from the directed-container laws). Results checked in those files carry the Lean-verified tag naming the committed file. Where MacBeth has a Lean formalisation that is *not* yet committed as source to this repository (notably the **DCont** $\cong$ **Cof** and Zappa–Szép associativity developments), the statement is tagged [Cited] for the underlying theorem and a footnote flags that the Lean source is not in this tree — so the reader is never told something is machine-checked here when its source cannot be found here.

Chapter 1

Containers and their extension functor

1.1 Containers

Definition 1.1 (Container)**.** A *container* is a pair $S \triangleleft P$ where S is a set of *shapes* and $P\colon S \to \mathbf{Set}$ is a family of *positions*, one set $P(s)$ for each shape s. Its *extension* (or *interpretation*) is the functor $[\![S \triangleleft P]\!]\colon \mathbf{Set} \to \mathbf{Set}$ defined on objects by

$$[\![S \triangleleft P]\!](X) \;=\; \coprod_{s \in S} X^{P(s)} \;=\; \big\{(s, v) \mid s \in S,\ v\colon P(s) \to X\big\},$$

and on a morphism $f\colon X \to Y$ by postcomposition, $[\![S \triangleleft P]\!](f)(s, v) = (s,\ f \circ v)$.

Unpacking: a shape s is a structure with its positions $P(s)$; a labelling $v\colon P(s) \to X$ fills every position with an element of X; the coproduct over s is the disjoint union "pick exactly one shape". The functorial action relabels — it changes the data at the positions but never the shape, so the *shape projection* $\pi(s, v) = s$ is a natural transformation to the constant functor at S. A container thus separates *structure* (the shape) from *content* (the labels), and its functorial action is exactly `map`.

Example 1.2 (The container menagerie)**. Maybe**: two shapes, "nothing" ($P = \varnothing$, term $X^0 = 1$) and "just" ($P = \{*\}$, term X); extension $1 + X$. **Lists**: shape $n \in \mathbb{N}$, positions $\{0, \dots, n-1\}$; extension $\sum_n X^n$. **Binary trees**: a shape is a planar binary tree, positions are leaves; the generating function satisfies $T(X) = 1 + X \cdot T(X)^2$, so the number of shapes with n positions is the Catalan number $C_n = \frac{1}{n+1}\binom{2n}{n}$. **Matrices**: shape (m, n), positions the mn entries; extension $\sum_{m,n} X^{m \times n}$. In each case the coefficient of X^n in the generating function counts *how many* shapes have n positions, while the container remembers *which* shapes and *where* the positions are.

A standing intuition ("decategorification"): writing $\sum_s X^{P(s)}$ as if it were a polynomial in a formal indeterminate X guides the right intuitions about products, composition and derivatives. But X is a *set*, not an indeterminate: $X^{P(s)} = \mathrm{Hom}(P(s), X)$ is a set of functions, and the extension lives in $[\mathbf{Set}, \mathbf{Set}]$, not in a ring of power series.

1.2 The representation theorem

The combinatorial data of containers captures *exactly* the natural transformations between their extensions.

Theorem 1.3 (Representation / full faithfulness)**.** *The extension functor* $[\![-]\!]\colon \mathbf{Cont} \to [\mathbf{Set}, \mathbf{Set}]$ *is full and faithful: every natural transformation* $[\![S_1 \triangleleft P_1]\!] \Rightarrow [\![S_2 \triangleleft P_2]\!]$ *arises from a unique container morphism, and distinct morphisms induce distinct natural transformations.* *[Cited: Abbott–Altenkirch–Ghani 2005]*[1]

Proof sketch. By the Yoneda lemma a natural transformation α is determined by its action on the "generic element" $(s, \mathrm{id}_{P_1(s)}) \in [\![S_1 \triangleleft P_1]\!](P_1(s))$ of each shape: $\alpha_{P_1(s)}(s, \mathrm{id}) = (u(s), f_s)$ reads off both a shape map $u\colon S_1 \to S_2$ and a position map $f_s\colon P_2(u(s)) \to P_1(s)$, and naturality forces every other component. Conversely every (u, f) is natural (Chapter 2). See [1] for the full argument; the round-trip is machine-checked in `Basic.lean`. □

[1] Independently formalised by MacBeth in the committed Lean file `lean/Containers/Containers/Basic.lean` (round-trip `ContainerMorphism` ↔ `ExtNatTrans`, no `sorry`).

Chapter 2

The category Cont

2.1 Container morphisms: the variance

Definition 2.1 (Container morphism)**.** A *container morphism* $(u, f)\colon S_1 \triangleleft P_1 \to S_2 \triangleleft P_2$ consists of

- a *shape map* $u\colon S_1 \to S_2$ (*forward*), and
- for each $s \in S_1$, a *position map* $f_s\colon P_2(u(s)) \to P_1(s)$ (*backward*).

It induces the natural transformation $\alpha_X(s, v) = \big(u(s),\, v \circ f_s\big)$.

The variance is not a convention but a necessity. To build a new labelling of the *output* positions $P_2(u(s))$ out of an old labelling $v\colon P_1(s) \to X$ without inspecting labels, the only move is to precompose:

$$P_2(u(s)) \xrightarrow{f_s} P_1(s) \xrightarrow{v} X,$$

which forces f_s to run from output positions to input positions. Each output position p' asks "which input position supplies my label?" and the answer is $f_s(p')$. Consequently a container morphism can only *rearrange*, *discard* or *duplicate* labels — never *create* one. (Sorting, filtering and thresholding are *not* container morphisms: they inspect labels, and so are not natural in X.)

Definition 2.2 (Identity and composition)**.** The identity on $S \triangleleft P$ is $(\mathrm{id}_S, \mathrm{id})$. The composite of $(u, f)\colon S_1 \triangleleft P_1 \to S_2 \triangleleft P_2$ and $(u', f')\colon S_2 \triangleleft P_2 \to S_3 \triangleleft P_3$ has shape map $u' \circ u$ and position map running backward through both stages,

$$(u' \circ u)^\sharp_s \;=\; f_s \circ f'_{u(s)}\colon P_3\big(u'(u(s))\big) \xrightarrow{f'_{u(s)}} P_2(u(s)) \xrightarrow{f_s} P_1(s).$$

These satisfy the category axioms; the resulting category is **Cont**.

Proposition 2.3 (Naturality and the correspondence)**.** *For every container morphism (u, f), the family $\alpha_X(s, v) = (u(s), v \circ f_s)$ is natural in X; and (Theorem 1.3) every natural transformation between container extensions is α for a unique (u, f). Hence $[\![-]\!]\colon$ **Cont** $\to [\mathbf{Set}, \mathbf{Set}]$ is an isomorphism onto its image.* *[Cited: AAG05]*

Proof. Naturality is associativity of composition: for $g\colon X \to Y$, both ways round the naturality square send (s, v) to $(u(s),\, g \circ v \circ f_s)$, because f_s does not look at labels. Full faithfulness is Theorem 1.3. □

2.2 Worked example: `tail`

The function $\mathrm{tail}\colon \mathrm{List}(X) \to 1 + \mathrm{List}(X)$ is a container morphism (u, f): the shape map sends length n to length $n - 1$ (and 0 to "nothing"), and the (backward) position map sends position

i of the output to position $i+1$ of the input. The same (u, f) acts on lists of integers, strings, or anything — the labels are shifted one place left, never inspected. This is the content of naturality: the morphism is parametric in the label type.

Chapter 3

Algebraic structure on Cont

This chapter is, by design, partly an *outline with cited statements*. The monoidal and closed structures on containers are most cleanly developed in the polynomial-functor setting, where **Cont** embeds as the full subcategory **Poly** of polynomial endofunctors. We record the structures, cite their sources, and flag carefully which MacBeth has personally verified. We do *not* reprove them here, and we do not invent proofs.

Remark 3.1 (Containers and polynomial functors). A container $S \triangleleft P$ presents the polynomial functor $X \mapsto \sum_{s \in S} X^{P(s)}$; the category **Poly** of polynomial functors and natural transformations is equivalent to **Cont** (this is Theorem 1.3 read as "**Poly** $\simeq$ **Cont**"). We use the two names interchangeably and adopt Spivak's catalogue of monoidal structures on **Poly** [4].

3.1 Products and coproducts

Proposition 3.2 (Coproducts in **Cont**)**. Cont** *has all coproducts: the coproduct of* $S_i \triangleleft P_i$ *has shape set* $\coprod_i S_i$ *and positions inherited shapewise,* $P\big((i,s)\big) = P_i(s)$*. On extensions it is the pointwise coproduct of functors,* $[\![\coprod_i C_i]\!] = \coprod_i [\![C_i]\!]$*.* *[Cited: AAG05; Spivak 2021]*

Proposition 3.3 (Products in **Cont**)**. Cont** *has all products: the product of* $S_1 \triangleleft P_1$ *and* $S_2 \triangleleft P_2$ *has shape set* $S_1 \times S_2$ *and positions* $P\big((s_1,s_2)\big) = P_1(s_1) \sqcup P_2(s_2)$*, so that* $[\![C_1 \times C_2]\!](X) = [\![C_1]\!](X) \times [\![C_2]\!](X)$*.* *[Cited: Spivak 2021]*[1]

3.2 The monoidal menagerie

Spivak [4] catalogues several monoidal structures on **Poly**. We list the ones relevant to this book; the reader should treat each as *cited, not independently checked* unless tagged otherwise.

Definition 3.4 (The four products, after Spivak 2021)**.** On **Poly** $\simeq$ **Cont** there are, among others:

- the *coproduct* $+$ (Prop. 3.2), unit the initial polynomial 0;
- the *product* $\times$ (Prop. 3.3), unit the terminal polynomial 1;
- the *Dirichlet (parallel) tensor* $\otimes$: on shapes $S_1 \times S_2$, on positions $P_1(s_1) \times P_2(s_2)$, so that $[\![C_1 \otimes C_2]\!](X) = \sum_{s_1,s_2} X^{P_1(s_1)\times P_2(s_2)}$; unit the identity polynomial $X \mapsto X$ $(1 \triangleleft 1)$;
- the *composition (substitution) product* $\triangleleft$, with $[\![C_1 \triangleleft C_2]\!] = [\![C_1]\!] \circ [\![C_2]\!]$; unit the identity polynomial. This is the *sequencing* product, and comonoids for it are the subject of Chapter 4.

[Cited: Spivak 2021]

[1] Stated as in [4]; the coproduct (Prop. 3.2) MacBeth has checked directly on extensions, the product MacBeth has *not* independently re-derived in full and reports it as cited.

Remark 3.5 (Honesty about $\otimes$ vs. $\times$). The symbols are easy to confuse. The *Dirichlet* tensor $\otimes$ multiplies *position sets* ($P_1 \times P_2$); the categorical *product* $\times$ takes a *disjoint union* of position sets ($P_1 \sqcup P_2$). They have the same shape set $S_1 \times S_2$ but different position data and different extensions. Both are due to Spivak's catalogue [4]; MacBeth has verified the shape/position formulas combinatorially on small cases but defers the unit and coherence laws to [4].

Definition 3.6 (Closed structures)**.** ($\mathbf{Poly}, \otimes$) and ($\mathbf{Poly}, \times$) are closed: each has an internal hom ("$[-,-]$") right adjoint to the corresponding tensor. The composition product $\triangleleft$ is *not* symmetric and its closure is subtler (the relevant right adjoints exist only in restricted forms). [Cited: Spivak 2021][2]

The free-monad and cofree-comonad constructions associated with $\triangleleft$ are deferred to Chapter 6, where they belong to the functors-over-**Cont** programme rather than to the structure of **Cont** itself.

[2]The internal homs are stated as in [4]; MacBeth has *not* independently verified the adjunctions and reports them as cited.

Chapter 4

Comonoids in Cont: directed containers and small categories

This is the heart of the book. We show that comonoids for the composition product $\triangleleft$ are *directed containers*, that directed containers are *small categories*, and — the refinement that the rest of the grant turns on — that the right *morphisms* are *cofunctors*, not functors.

4.1 Directed containers

Definition 4.1 (Directed container)**.** A *directed container* is a container $S \triangleleft P$ together with

- a *root* $\mathsf{o}(s) \in P(s)$ for each s;
- a *sub-shape* operation $s \downarrow p \in S$ for $p \in P(s)$;
- a *shift* $p \oplus q \in P(s)$ for $p \in P(s)$, $q \in P(s \downarrow p)$;

subject to the five laws (for all s, $p \in P(s)$, $q \in P(s \downarrow p)$, $q' \in P((s \downarrow p) \downarrow q)$):

$$\text{(D1)}\quad s \downarrow \mathsf{o}(s) = s; \qquad \text{(D2)}\quad \mathsf{o}(s) \oplus q = q; \qquad \text{(D3)}\quad p \oplus \mathsf{o}(s \downarrow p) = p;$$
$$\text{(D4)}\quad s \downarrow (p \oplus q) = (s \downarrow p) \downarrow q; \qquad \text{(D5)}\quad (p \oplus q) \oplus q' = p \oplus (q \oplus q').$$

D2, D3, D5 make each $P(s)$ a monoid-like structure with unit $\mathsf{o}(s)$; D1, D4 track how sub-shapes nest. (The well-typedness of D5 needs D4 — exactly the dependency that recurs as a dependent-cast obstacle in the Lean formalisation.)

4.2 Directed containers are comonads

Definition 4.2 (The comonad of a directed container)**.** On $D := [\![S \triangleleft P]\!]$ define

$$\varepsilon(s, v) = v(\mathsf{o}(s)), \qquad \delta(s, v) = \big(s,\ \lambda p.\,(s \downarrow p,\ \lambda q.\, v(p \oplus q))\big).$$

Theorem 4.3 (Comonad laws $\iff$ D1–D5)**.** *(D, ε, δ) is a comonad, and each comonad law is exactly one directed-container law: left counit is D1 (shape) and D2 (value); right counit is D3; coassociativity is D4 (shapes) and D5 (deepest values). Conversely a comonad of this form forces D1–D5.* *[Cited: Ahman–Chapman–Uustalu 2014; MacBeth, Lean-verified: Directed.lean]*

Proof sketch. Direct calculation. For left counit, $\varepsilon(\delta(s, v))$ reduces to $(s \downarrow \mathsf{o}(s),\ \lambda q.\, v(\mathsf{o}(s) \oplus q))$, which is (s, v) by D1 then D2. Right counit is D3; coassociativity unfolds both composites to a common triple nest, equal by D4 (inner shapes) and D5 (deepest values). The full computation, and the converse, are checked with zero `sorry` in the committed file `lean/Containers/Containers/Directed.l` (`left_counit`, `right_counit`, `coassoc`); the "container is a comonad" direction is Ahman–Chapman–Uustalu [2]. □

4.3 Directed containers are small categories

Theorem 4.4 (Object dictionary)**.** *Directed-container structures on $S \triangleleft P$ are in bijection with small categories $\mathcal{C}$ with object set S and, for each s, $P(s) = \coprod_{s'} \mathcal{C}(s, s')$ the morphisms* out of *s, under*

$$\operatorname{ob}\mathcal{C} = S, \quad \mathcal{C}(s, s') = \{p \in P(s) : s \downarrow p = s'\}, \quad \mathrm{id}_s = \mathsf{o}(s), \quad q \circ p = p \oplus q.$$

In particular $s \downarrow p = \mathrm{cod}(p)$, and every small category arises from a unique directed container. *[Cited: Ahman–Uustalu 2016]*

Combining Theorems 4.3 and 4.4 with the polynomial incarnation gives the object-level chain promised in the Preface: a directed container is a comonoid in $(\mathbf{Poly}, \triangleleft)$, hence

Containers $\supseteq$ **Directed Containers** $\cong$ **Polynomial Comonoids** $\cong$ **Small Categories**.

4.4 The morphism refinement: DCont $\cong$ Cof

A dictionary on objects invites a dictionary on morphisms. The naive guess — a morphism of directed containers is a *functor* — is *false on variance*. A container morphism has a backward position map $f^\sharp_s \colon P'(fs) \to P(s)$, whereas a functor pushes morphisms forward. The maps native to directed containers are *cofunctors*.

Definition 4.5 (Cofunctor / retrofunctor)**.** A *cofunctor* $\varphi \colon \mathcal{C} \leftharpoonup \mathcal{D}$ is an object map $\varphi_0 \colon \operatorname{ob}\mathcal{C} \to \operatorname{ob}\mathcal{D}$ together with a *lift*: for each object c and each $h \colon \varphi_0 c \to d$ in $\mathcal{D}$, a morphism $\varphi^\uparrow(c, h) \colon c \to c'$ in $\mathcal{C}$, subject to

$$\text{(C0) } \varphi_0(\mathrm{cod}\varphi^\uparrow(c,h)) = \mathrm{cod}(h); \quad \text{(C1) } \varphi^\uparrow(c, \mathrm{id}_{\varphi_0 c}) = \mathrm{id}_c;$$
$$\text{(C2) } \varphi^\uparrow(c, h' \circ h) = \varphi^\uparrow(c', h') \circ \varphi^\uparrow(c, h), \quad c' = \mathrm{cod}\varphi^\uparrow(c, h).$$

Small categories and cofunctors form a category **Cof**. [Cited: Aguiar 1997; Clarke 2020; Spivak 2021 (Def. 3.21)]

Remark 4.6 (**Cof** $= \mathbb{C}\mathrm{at}^\sharp$)**.** **Cof** is exactly $\mathbb{C}\mathrm{at}^\sharp = \mathrm{Comod}(\mathbf{Poly})$, the category of comonoids in $(\mathbf{Poly}, \triangleleft)$ and their morphisms: a cofunctor *is* a comonoid homomorphism [4, 6]. This is why the object dictionary (Theorem 4.4) cannot extend to **Cat**: functors are carried not by comonoid morphisms but by bicomodules [5].

Definition 4.7 (Directed-container morphism)**.** A *morphism of directed containers* $D \to D'$ is a container morphism $(f, f^\sharp)$ with $f \colon S \to S'$ and $f^\sharp_s \colon P'(fs) \to P(s)$, satisfying, for all s, $h \in P'(fs)$, $k \in P'((fs) \downarrow' h)$,

$$\text{(M0) } f(s \downarrow f^\sharp_s(h)) = (fs) \downarrow' h; \quad \text{(M1) } f^\sharp_s(\mathsf{o}'(fs)) = \mathsf{o}(s);$$
$$\text{(M2) } f^\sharp_s(h \oplus' k) = f^\sharp_s(h) \ \oplus \ f^\sharp_{s \downarrow f^\sharp_s(h)}(k).$$

Theorem 4.8 (The morphism dictionary)**.** *The assignment $(f, f^\sharp) \mapsto (\varphi_0 := f, \ \varphi^\uparrow(s,h) := f^\sharp_s(h))$ is a bijection on every hom-set carrying (M0)–(M2) to (C0)–(C2) term for term, and it respects identities and composition. It is moreover a bijection on objects (Theorem 4.4). Hence there is an isomorphism of categories*

$$\mathbf{DCont} \ \cong \ \mathbf{Cof}.$$

[Cited: Ahman–Uustalu 2016 §3.2–3; Shapiro–Spivak 2023, Cor. 5.12][1]

[1] MacBeth has formalised this isomorphism in Lean (file `Cofunctor.lean`, built and reported zero-`sorry` in `papers/dcont-cof.tex`), but that source is *not yet committed* to this repository's `lean/` tree; so the statement is tagged [Cited] here, not Lean-verified, pending the source landing in the repo. The *identification* of Ahman–Uustalu's "relative split pre-opcleavages" with Spivak's cofunctors, and the bridge to delta lenses below, are MacBeth's connective contribution; see `papers/dcont-cof.tex`.

Proof sketch. Under the object dictionary a position $h \in P'(fs)$ is a morphism out of fs and $f^\sharp_s(h)$ a morphism out of s, so $(f, f^\sharp)$ and $(\varphi_0, \varphi^\uparrow)$ are literally the same data. Reading $\oplus$ as reverse composition, (M0)⇔(C0) (codomain), (M1)⇔(C1) (unit), (M2)⇔(C2) (composition). Identities and composites match, and objects are in bijection by Theorem 4.4; a bijective-on-objects, fully faithful functor is an isomorphism of categories. The full term-by-term match is in `papers/dcont-cof.tex` (Lemma 2.5–Theorem 2.7 there). □

Proposition 4.9 (Functors are the opposite variance)**.** *The* covariant *position maps* $\phi_s \colon P(s) \to P'(fs)$ *subject to the mirror conditions (N0)–(N2) of (M0)–(M2) are in bijection with* functors $\mathcal{C}(D) \to \mathcal{C}(D')$*. Thus a directed structure carries two notions of map of opposite variance: covariant gives functors, contravariant gives cofunctors, and only the latter is a morphism of the underlying container.* *[MacBeth]*[2]

Corollary 4.10 (Cofunctors are the Put of a delta lens)**.** *A* delta lens $A \rightleftarrows B$ *is a* Get *functor together with a* Put *cofunctor sharing one object map, related by the section condition (PutGet)* $f \cdot \varphi(a, u) = u$*. Under* **DCont** ≅ **Cof** *the Put cofunctor is exactly a morphism of directed containers; so a directed-container morphism is the update-propagating half of a bidirectional transformation.* *[Cited: Clarke 2022 (arXiv:2108.00390), Def. 4 / Thm. 9]*[3]

[2]Verified combinatorially over 36 ordered pairs of small categories drawn from $\{\mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbb{Z}/2, \mathbb{Z}/3, M\}$; #DCont-morphisms = #cofunctors in all 36 cases, #op-variance-morphisms = #functors in all 36, and #functors ≠ #cofunctors in 17 of the 36. See `papers/dcont-cof.tex` §5. The underlying fact that functors are carried by bicomodules, not comonoid morphisms, is [5].

[3]The section condition is (PutGet) $f \cdot \varphi(a, u) = u$, *not* "$f \circ \varphi = \mathrm{id}$"; MacBeth flags this because the loose form recurs in the folklore.

Chapter 5

Composing systems: Zappa–Szép and distributive laws

Chapter 4 settled what a single directed container is and what maps it has. This chapter asks how *two* of them compose into one — the question with the economic consequences (mis-composed supply chains lose liability traceability; a wrong contract composite is an exploit). We package this as a *Zappa–Szép* (two-sided semidirect / matched-pair) product of small categories, the categorical generalisation of the group/monoid Zappa–Szép product.

5.1 The pairwise criterion

For objects a, b of a small category, $\mathrm{Hom}(a, b)$ is canonically a right $\mathrm{End}(a)$-set (precomposition) and a left $\mathrm{End}(b)$-set (postcomposition). A Zappa–Szép decomposition $K = \mathcal{C} \bowtie \mathcal{D}$ of K over a wide subcategory $\mathcal{D}$ asks for a *unique factorisation* of each morphism through a transversal and an endomorphism.

Theorem 5.1 (Pairwise Zappa–Szép criterion)**.** *Let $\mathcal{D}$ be a wide subcategory of K. Then $K = \mathcal{C} \bowtie \mathcal{D}$ for a complementary wide subcategory $\mathcal{C}$ if and only if*

- (L) *each hom-presheaf* $\mathrm{Hom}_K(-, b)$ *is a* free *right $\mathcal{D}$-set (equivalently: the source-oriented factorisation $f = t \circ e$ is unique for every f), and*
- (G) *the free bases close into a wide subcategory $\mathcal{C}$.*

[MacBeth][1]

Theorem 5.2 (Cocycle axioms $\Longleftrightarrow$ associativity)**.** *The Zappa–Szép cocycle axioms ZS1–ZS4 (relating the two actions of a matched pair) hold if and only if the product on $\mathcal{C} \bowtie \mathcal{D}$ is associative.* *[MacBeth]*[2]

5.2 (G) as a strict factorisation system, and its H^2 obstruction

Proposition 5.3 ((G) is a strict factorisation system)**.** *Condition (G) — that the free bases close into a wide subcategory — is exactly the statement that the decomposition extends to a*

[1] This is the corrected form of the SEED conjecture "every small category is a Zappa–Szép product of a poset skeleton and its vertex monoids," which was *refuted* computationally (smallest counterexample: two objects, one arrow $s\colon a \to b$, $\mathrm{End}(a) = \mathrm{End}(b) = \mathbb{Z}/2$ fixing s, so $2 \nmid 1 = |\mathrm{Hom}(a, b)|$). The freeness half (L) is written up self-contained in `papers/pairwise-zappa-szep.tex` (a built PDF is in the repo; the `.tex` source is not yet committed to this tree). A brute-force enumeration of ≈ 26 small categories matches the criterion exactly.

[2] MacBeth has a Lean formalisation of the monoid case (ZS1–ZS4 $\Longleftrightarrow$ associativity of the Zappa–Szép product) in `ZappaSzep.lean`, built and reported zero-`sorry`; that source is *not yet committed* to this repository's `lean/` tree, so the tag is [MacBeth] rather than Lean-verified here.

strict factorisation system *on K. Distributive laws of categories correspond to strict factorisation systems.* *[Cited: Rosebrugh–Wood (distributive laws $\Longleftrightarrow$ SFS)]*

Theorem 5.4 (The (G) obstruction is an H^2 class)**.** *Under a trivial-action hypothesis, (G) is classified by a second cohomology class:* (G) $\Longleftrightarrow$ $[\omega] = 0$*, where* $[\omega]$ *lives in the* Baues–Wirsching *cohomology* $H^2(\mathrm{Sk}_{\mathcal{C}}; \mathcal{D})$ *of the $\mathcal{C}$-skeleton with a source-only (presheaf) natural system of coefficients. This is the split case of the Baues–Wirsching linear-extension classification.* *[MacBeth (identification), resting on Cited: Baues–Wirsching 1985]*[3]

5.3 The classical anchor

Proposition 5.5 (Classical Zappa–Szép recovered)**.** *Specialising Theorem 5.1 to a one-object category (a monoid) recovers the classical Zappa–Szép product of monoids, and (L) becomes the familiar condition that the factorisation map is a bijection.* *[MacBeth]*[4]

[3] The cohomological *identification* — that the obstruction to (G) is a Baues–Wirsching H^2 class and that "(G)$\Leftrightarrow$ $[\omega] = 0$" is the split case of the BW linear-extension theorem — is MacBeth's; the underlying classification theorem is Baues–Wirsching, *Cohomology of small categories*, JPAA **38** (1985) 187–211 (modern account: arXiv:2508.00727). The rigid twist of the counterexample above is the $\mathbb{Z}/2$ generator; the nonabelian boundary is the Kac/Rosebrugh–Wood law.

[4] MacBeth flags an earlier *over*claim for correction: the "Ahman–Uustalu update monad as a degenerate Zappa–Szép product" reading is *not* a special case of the two-category matched pair — it is a monoid acting on a set, not two composable monoids — and should not be presented as such.

Chapter 6

Functors over the category of containers (Phase 2 — outline)

This chapter is *mostly* a stub sketching Neil's proposed Phase 2 programme; the one exception is §**??**, which contains worked, small-case-verified results on the free monad and cofree comonad. Where Chapters 1–5 studied containers and the maps *between* them, Phase 2 studies *endofunctors of* **Cont** — operations $\mathbf{Cont} \to \mathbf{Cont}$ (equivalently $\mathbf{Poly} \to \mathbf{Poly}$) and the structures they carry.

What a functor $\mathbf{Cont} \to \mathbf{Cont}$ is. It assigns to each container a container, functorially in container morphisms. The programme is to identify the natural such operations and the (co)monad structures they support.

Candidate examples (to be developed, not yet results).

- *The derivative / one-hole-context functor* ∂: sends $S \triangleleft P$ to the container of "a shape with one position singled out". For lists, $\partial(\sum_n X^n)$ is the zipper. The combinatorial derivative is classical for species; its container-level naturality and its interaction with $\triangleleft$ are the targets here.
- *The free monad functor* on $(\mathbf{Poly}, \triangleleft)$ and the dual *cofree comonad functor*: the syntax/behaviour pair. These two are developed concretely in §**??** below — their shape/position data and (co)monad laws are worked out and small-case verified, so unlike the derivative bullet they are *results*, not stubs.

6.1 The free monad and cofree comonad of a container

We work where W- and M-types exist (e.g. **Set**). For a container $C = S \triangleleft P$ both the free monad and the cofree comonad of $[\![C]\!]$ are again containers; we give their shape/position data explicitly and reduce the (co)monad laws to the monoid laws of *tree grafting* and *path concatenation*. Throughout, $I = 1 \triangleleft 1$ is the $\triangleleft$-unit.

Definition 6.1 (Free monad container)**.** $C^* = S^* \triangleleft P^*$ where $S^* = \mu Y.\,\big(1 + \sum_{s:S}(P(s) \to Y)\big)$ is the set of well-founded C-trees with a distinguished *variable-leaf* constructor (the 1), and $P^*(w)$ is the set of variable leaves of w: $P^*(\bullet) = 1$, $P^*(\mathsf{node}(s, c)) = \sum_{p:P(s)} P^*(c(p))$.

Definition 6.2 (Cofree comonad container)**.** $C^\infty = S^\infty \triangleleft P^\infty$ where $S^\infty = \nu Y.\,\sum_{s:S}(P(s) \to Y)$ is the M-type of (possibly infinite) C-trees, and $P^\infty(w)$ is the set of *nodes* of w, i.e. finite paths from the root: $P^\infty(\langle s, c\rangle) = 1 + \sum_{p:P(s)} P^\infty(c(p))$ (the empty path $\bullet$ plus the paths into each child). Each path is finite even when w is infinite.

Proposition 6.3 (Extensions)**.** $[\![C^*]\!]A \cong \mu X.\,(A + [\![C]\!]X)$ *is the carrier of the free monad on* $[\![C]\!]$*, and* $[\![C^\infty]\!]A \cong \nu X.\,(A \times [\![C]\!]X)$ *is the carrier of the cofree comonad on* $[\![C]\!]$ *— the possibly-infinite* C*-trees with* every node *labelled in* A*.* *[Cited: Abbott–Altenkirch–Ghani 2005 (W/M-types); Spivak 2021 (tree polynomial). Container-language derivation re-done here; small cases below.]*

The structure maps are container morphisms. *Unit* $\eta : I \to C^*$ sends $*$ to the variable leaf $\bullet$. *Multiplication* $\mu : C^* \triangleleft C^* \to C^*$ grafts the tree $g(p)$ into each variable leaf p of w; on positions it is the canonical iso $P^*(\mathsf{graft}(w, g)) \cong \sum_{p:P^*(w)} P^*(g(p))$, i.e. the identity. Dually, *counit* $\varepsilon : C^\infty \to I$ has $f_\varepsilon(w)(*) = \bullet$ (the root), and *comultiplication* $\delta : C^\infty \to C^\infty \triangleleft C^\infty$ has $u_\delta(w) = (w, \lambda n.\, w/n)$ (label each node n by the subtree w/n) and $f_\delta(w)(n, m) = n \cdot m$ (path concatenation).

Theorem 6.4 ((Co)monad laws via grafting and concatenation)**.** (C^*, η, μ) *is a monoid and* $(C^\infty, \varepsilon, \delta)$ *a comonoid in* $(\mathbf{Cont}, \triangleleft)$*. The monad laws reduce to the monoid laws of grafting (the variable leaf is neutral; substitution is associative); the comonad laws reduce to those of path concatenation (the root is the empty path; concatenation is associative).* *[MacBeth, 2026-06-11: laws checked by exhaustive enumeration of small labelled trees for* $C = 1 + X$ *(Maybe) and* $C = 1 + X^2$ *(binary); scripts in* `projects/scratch/`*.]*

Theorem 6.5 (C^∞ is a directed container)**.** C^∞ *is a directed container with root* $\mathsf{o}(w) = \bullet$*, sub-shape* $w \downarrow n = w/n$ *(the subtree at node* n*), and shift* $n \oplus m = n \cdot m$*. Laws* D1–D5 *(Def. 4.1) hold, the only non-formal one being* $w/(n \cdot m) = (w/n)/m$ *behind* D4*. By Theorem 4.3 this directed container* is *the comonad above: the cofree comonad of a container is the cofree* directed *container on it. Its category (Theorem 4.4) is the* subtree category*: objects are trees, a morphism* $w \to w'$ *is a node* n *of* w *with* $w/n = w'$*, identities are roots, composition is concatenation.* *[Cited in substance: Ahman–Chapman–Uustalu (polynomial comonad = category); Spivak, and Spivak–Niu* Polynomial Functors *(cofree comonoid = tree comonoid). MacBeth: explicit* $\mathsf{o}/\downarrow/\oplus$ *data in the D1–D5 form and the verification.]*

Example 6.6 (Maybe and binary)**.** For $C = 1 + X$ (shapes nil with $P = \varnothing$, cons with $P = 1$): $S^\infty = \nu Y.\, 1 + Y = \bar{\mathbb{N}}$ (conaturals) and $[\![C^\infty]\!]A$ is the nonempty colists of A — the classical $\mathsf{head}/\mathsf{tails}$ comonad; while $[\![C^*]\!]A$ is the free monad on Maybe (lists terminated by a variable or nil). For the binary container $C = 1 + X^2$, C^* is finite binary trees with variable leaves and C^∞ is possibly-infinite A-labelled binary trees, δ relabelling each node by its subtree.

Why it matters for the grant. Free monads and cofree comonads are the syntax/behaviour pair for compositional systems: a system's available moves (free monad) and its observable unfolding (cofree comonad). Verified composition at this level would let one certify, e.g., that an agent-orchestration combinator preserves the lens (Put/Get) structure of its parts. This is flagged as the *next direction*; nothing in this chapter is claimed as proved.

Appendix A

Reading & citation tracker

The status column records how thoroughly MacBeth has engaged each source: "read in full" (worked through, definitions and key proofs internalised), "skimmed" (read for the relevant statement), or "to-read".

Paper	**arXiv / ref**	**Status**	**Used in**
Abbott–Altenkirch–Ghani, *Containers: constructing strictly positive types* (2005)	TCS 342	read in full	Ch. 1–2
Ahman–Chapman–Uustalu, *When is a container a comonad?* (2014)	LMCS 10(3)	read in full	Ch. 4
Ahman–Uustalu, *Directed containers as categories* (2016)	arXiv:1604.01187	read in full	Ch. 4
Spivak, *Functorial aggregation / Polynomial functors* (2021)	arXiv:2111.10968	read in full	Ch. 3–4
Shapiro–Spivak, *Structures on categories of polynomials* (2023)	arXiv:2305.00167	read in full	Ch. 4
Shapiro–Spivak, *A polynomial construction of nerves* (2024)	arXiv:2405.13157	skimmed	Ch. 4
Clarke, *Internal lenses as functors and cofunctors* (2020)	arXiv:2009.06835	read in full	Ch. 4
Clarke, *Delta lenses as coalgebras for a comonad* (2022)	arXiv:2108.00390	read in full	Ch. 4
Rosebrugh–Wood, *Distributive laws and factorization* (JPAA 175, 2002)	JPAA 175	read in full	Ch. 5
Baues–Wirsching, *Cohomology of small categories* (1985)	JPAA 38, 187–211	read in full	Ch. 5
(modern BW account) (2025)	arXiv:2508.00727	skimmed	Ch. 5
Aguiar, *Internal categories and quantum groups* (1997)	PhD, Cornell	skimmed	Ch. 4

Bibliography

[1] M. Abbott, T. Altenkirch, N. Ghani. *Containers: constructing strictly positive types*. Theoretical Computer Science **342** (2005) 3–27.

[2] D. Ahman, J. Chapman, T. Uustalu. *When is a container a comonad?* Logical Methods in Computer Science **10**(3), 2014.

[3] D. Ahman, T. Uustalu. *Directed containers as categories*. Proc. MSFP 2016, EPTCS **207**, 89–98. arXiv:1604.01187.

[4] D. I. Spivak. *Functorial aggregation* (polynomial functors). arXiv:2111.10968, 2021.

[5] B. Shapiro, D. I. Spivak. *Structures on categories of polynomials*. arXiv:2305.00167, 2023.

[6] B. Shapiro, D. I. Spivak. *A polynomial construction of nerves for higher categories*. arXiv:2405.13157, 2024.

[7] B. Clarke. *Internal lenses as functors and cofunctors*. Proc. ACT 2019, EPTCS **323**, 183–195. arXiv:2009.06835.

[8] B. Clarke. *Delta lenses as coalgebras for a comonad*. arXiv:2108.00390, 2022.

[9] M. Aguiar. *Internal categories and quantum groups*. PhD thesis, Cornell University, 1997.

[10] R. Rosebrugh, R. J. Wood. *Distributive laws and factorization*. Journal of Pure and Applied Algebra **175** (2002) 327–353.

[11] H.-J. Baues, G. Wirsching. *Cohomology of small categories*. Journal of Pure and Applied Algebra **38** (1985) 187–211.

[12] *(Modern account of Baues–Wirsching cohomology)*. arXiv:2508.00727, 2025.